

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

**Analýza zvukového obsahu prehrávaného v
prostredí OpenLab**

Bakalárska práca

2024

Nazar Aleksanyh

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

Analýza zvukového obsahu prehrávaného v prostredí OpenLab

Bakalárska práca

Študijný program:	Informatika
Študijný odbor:	9.2.1. Informatika
Školiace pracovisko:	Katedra počítačov a informatiky (KPI)
Školiteľ:	Ing. Matúš Sulír, PhD.
Konzultant:	Ing. Tomáš Kormaník

Košice 2024

Nazar Aleksanyh

Abstrakt v SJ

Táto práca sa zaoberá návrhom a implementáciou kontajnerového systému na analýzu zvuku v prostredí OpenLab. Naším cieľom bolo vytvoriť univerzálny systém, ktorý by sa dal prispôbiť pre rôzne aplikácie. Analytická časť práce bola zameraná na skúmanie metód spracovania zvuku zachyteného mikrofónmi. Skúmali sme rôzne techniky extrakcie príznakov a skúmali sme rôzne modely strojového učenia schopné vykonávať všeobecné úlohy klasifikácie zvuku. Na základe našich zistení sme navrhli systém pozostávajúci z dvoch častí. Serverová zložka je zodpovedná za streamovanie zvukových údajov v prostredí OpenLab a klientska zložka je určená na nahrávanie zvuku zo streamu, analýzu streamovaného zvuku pomocou zvoleného modelu a prenos výsledkov klasifikácie do MQTT brokera. Systém preukázal perspektívne výsledky a disponuje flexibilitou, ktorá sa dá prispôbiť na použitie v rôznych prostrediach.

Kľúčové slová v SJ

OpenLab, programovanie, zvuk, analýza, python

Abstrakt v AJ

This thesis explores the design and implementation of a containerized system for audio analysis within the OpenLab environment. Our goal was to create a versatile system that could be adapted for various applications. The analytic part of thesis was focused on examining methods for processing audio captured by microphones. We investigated different techniques for feature extraction and explored various machine learning models capable of performing general sound classification tasks. Based on our findings, we designed a two-part system. The server component is responsible for streaming audio data within the OpenLab environment and the client component is designed to record audio from stream, analyze the streamed audio using the chosen model, and transmit the classification results to an MQTT broker. The system has demonstrated promising results and possesses the versatility to be adapted for use in various environments.

Kľúčové slová v AJ

OpenLab, programming, audio, analysis, python

Bibliografická citácia

ALEKSANYCH, Nazar. *Analýza zvukového obsahu prehrávaného v prostredí OpenLab*. Košice: Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, 2024. 35s. Vedúci práce: Ing. Matúš Sulír, PhD.

TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY
Katedra počítačov a informatiky

Z A D A N I E
B A K A L Á R S K E J P R Á C E

Študijný odbor: **Informatika**

Študijný program: **Informatika**

Názov práce:

Analýza zvukového obsahu prehrávaného v prostredí OpenLab

Analysis of Audio Content Played in OpenLab

Študent: **Nazar Aleksanyh**

Školiteľ: **Ing. Matúš Sulír, PhD.**

Školiace pracovisko: **Katedra počítačov a informatiky**

Konzultant práce: **Ing. Tomáš Kormaník**

Pracovisko konzultanta: **Katedra počítačov a informatiky**

Pokyny na vypracovanie bakalárskej práce:

1. Analyzovať možnosti spracovania zvukového obsahu získaného z laboratória OpenLab.
2. Na základe analýzy navrhnúť kontajnerizovaný systém na spracovanie prijímaných dát.
3. Vyhodnotiť úspešnosť implementácie a presnosť výslednej analýzy.
4. Vypracovať dokumentáciu výsledného systému.

Jazyk, v ktorom sa práca vypracuje: slovenský

Termín pre odovzdanie práce: 24.05.2024

Dátum zadania bakalárskej práce: 31.10.2023




prof. Ing. Liberios Vokorokos, PhD.
dekan fakulty

Čestné vyhlásenie

Vyhlasujem, že som záverečnú prácu vypracoval(a) samostatne s použitím uvedenej odbornej literatúry.

Košice, 24.5.2024

.....

Vlastnoručný podpis

Podakovanie

Chcel by som poďakovať svojim najbližším priateľom a rodine za podporu počas celej mojej cesty na univerzitu.

Taktiež ďakujem konzultantovi *Ing. Tomášovi Kormaníkovi* za pomoc pri písaní bakalárskej práce.

Obsah

Úvod	2
1 Analytická časť	4
1.1 Digitálny zvuk	4
1.2 ffmpeg	5
1.3 Programovací jazyk Python	5
1.3.1 Knižnice pre vedecké výpočty a analýzu zvuku	6
1.3.2 Knižnice pre strojové učenie	6
1.3.3 Iné knižnice	6
1.4 PANNs	7
1.5 AST: Audio Spectrogram Transformer	8
1.6 musicnn	9
1.7 Docker	10
2 Syntetická časť	11
2.1 Implementácia servera	11
2.1.1 Inicializácia servera	11
2.1.2 Parametre audia	12
2.1.3 Konfigurácia audio vstupu	13
2.1.4 Získanie zvuku z mikrofónu	14
2.1.5 HTTP Live Streaming	15
2.1.6 Vytvorenie kontajnera Docker so serverom	16
2.2 Implementácia klienta	16
2.2.1 Porovnanie AST a PANNs	16
2.2.2 Konfiguračný súbor	17
2.2.3 Získanie zvukového streamu	18
2.2.4 Odosielanie správ do MQTT	20
2.2.5 Analýza zvuku	21
2.2.6 Vytvorenie kontajnera Docker s klientom	25

3	Vyhodnotenie	26
3.1	Vyhodnotenie servera	26
3.2	Vyhodnotenie klienta	27
3.2.1	Vyhodnotenie presnosti výslednej analýzy	27
4	Záver	30
	Literatúra	32
	Zoznam skratiek	36
	Zoznam príloh	37
A	Systémová príručka	38
A.1	Vývojové prostredie	38
A.2	Nasadenie systému	38
A.2.1	Nasadzovanie servera	38
A.2.2	Nasadzovanie klienta	39
A.3	Popis zdrojového kódu	39
A.3.1	Funkcie servera	39
A.3.2	Funkcie klienta	40
A.4	Obmedzenia	41

Zoznam obrázkov

1.1	A-D-A tok	4
1.2	Proces konverzie pomocou ffmpeg	5
1.3	Algoritmus PANNs	7
1.4	Algoritmus AST	8
1.5	Algoritmus musicnn	9
1.6	Architektúra Docker-a	10
2.1	Testovacia stránka	12
2.2	Flask po otvorení stránky	12
2.3	Výsledok s dostupnými vstupnými zvukovými zariadeniami	13
2.4	Konverzia zvuku pre streamovanie pomocou ffmpeg v termináli . .	15
2.5	Nahrávanie streamu v klientovi pomocou ffmpeg	20
3.1	Graf s presnosťou všeobecnej klasifikácie skladieb	28
3.2	Graf s presnosťou klasifikácie reči a konverzie na text	29

Zoznam zdrojových kódov

2.1	Inicializácia skúšobnej stránky	11
2.2	Kód na vyhľadávanie vstupných audio zariadení	13
2.3	Funkcia read_config	13
2.4	Funkcie response a read_audio	14
2.5	Funkcia audio_feed	15
2.6	Výstup AST	17
2.7	Výstup PANNs	17
2.8	Konfiguračný súbor pre klienta	17
2.9	Funkcia main	18
2.10	Funkcia download_hls_stream	19
2.11	Funkcia send_to_mqtt	20
2.12	Funkcia analyze_audio	21
2.13	Príklad správy zo všeobecnej klasifikácie	22
2.14	Funkcia trscrb_audio	22
2.15	Asynchrónna funkcia find_music	23
2.16	Príklad výsledku vyhľadania skladby	23
2.17	Funkcia analyze_music	24
2.18	Príklad správy po analýze hudby	24
2.19	Kód súboru Dockerfile	25

Úvod

OpenLab¹ je moderné laboratórium na chodbe Katedry počítačov a informatiky Technickej univerzity v Košiciach, ktoré nám poskytuje množstvo zariadení a ich možností na vývoj zaujímavých vecí a ich využitie na výskum. Jedným z nich sú priestorové mikrofóny, ktoré momentálne nemajú veľa možností využitia a my sa ich pokúsime vytvoriť.

Našou hlavnou úlohou v tejto práci je implementovať systém, ktorý rozšíri možnosti používania mikrofónov ako analyzovať zvuk v OpenLabe, aby nám poskytol informáciu o prehrávanom zvuku a aby študenti mali možnosť využiť tieto údaje na budúce projekty. Z informácií poskytnutých z katedry naše mikrofóny môžu len rozpoznávať hlas po kľúčovom slove “Hej Ola” pre virtuálnej asistentke Ola, aby študenti mali možnosť hlasového ovládania.

Na strane služieb môžeme použiť MQTT broker OpenLabu na odosielanie údajov a ich použitie na štatistiky a/alebo pre iné aplikácie. Zároveň použijeme Docker², aby sme mali možnosť streamovať zvuk z rôznych mikrofónov, analyzovať ich príslušné výstupy a posilať dáta do rôznych tém MQTT.

V súčasnosti máme veľa metód na analýzu zvuku, ktoré používajú rôzne algoritmy, datasety a spôsoby, ako nám poskytnúť výsledok, takže niektoré z nich otestujeme a vyberieme metódu, ktorá poskytne rýchlu analýzu a lepšie, presnejšie výsledky pre náš systém.

Po poskytnutí informácií o zvuku z mikrofónu máme možnosť zistiť, čo môžeme urobiť potom. Napríklad, ak zvuk obsahuje reč, môžeme ho previesť na text, v inom prípade, ak obsahuje hudbu, môžeme nájsť skladbu, ak je to možné, a analyzovať ju vlastnosti.

¹<https://kpi.fei.tuke.sk/sk/openlab>

²<https://www.docker.com/get-started/>

Formulácia úlohy

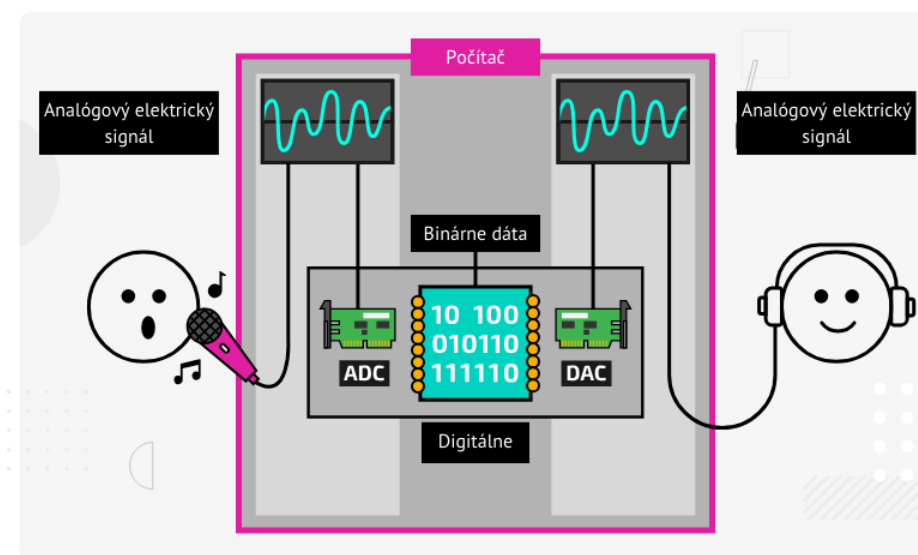
Vytvoríme obrázky Docker, jeden na streamovanie zvuku z mikrofónu, druhý na jeho analýzu. To nám zabezpečí izoláciu procesov a ich jednoduchú konfiguráciu. V časti analýzy zvuku robíme klasifikáciu zvuku. Ak výsledky ukazujú, že zvuk obsahuje reč, výstup sa skonvertuje na text. V prípade, že obsahuje hudbu, snažíme sa napísať, kto je spevák, názov piesne a analyzovať vlastnosti tejto hudby. Po všetkých týchto procesoch budú údaje odoslané na MQTT broker.

1 Analytická časť

V tejto kapitole budeme analyzovať, ako sa spracováva digitálny zvuk a aký programovací jazyk použijeme na napísanie systému. Taktiež budeme analyzovať existujúce modely na vytvorenie všeobecnej klasifikácie prehrávaného obsahu.

1.1 Digitálny zvuk

Digitálny zvuk je reprezentácia zvukových vln v digitálnom formáte pomocou série čísel, ktoré predstavujú amplitúdu pôvodnej zvukovej vlny pri každej vzorke (sample).[1] Digitálny zvuk sa používa všade v živote, napríklad v nahrávaní a reprodukcii hudby, filmová zvuková produkcia, rozpoznávanie reči, atď.



Obr. 1.1: Analógovo-digitálno-analógový tok¹

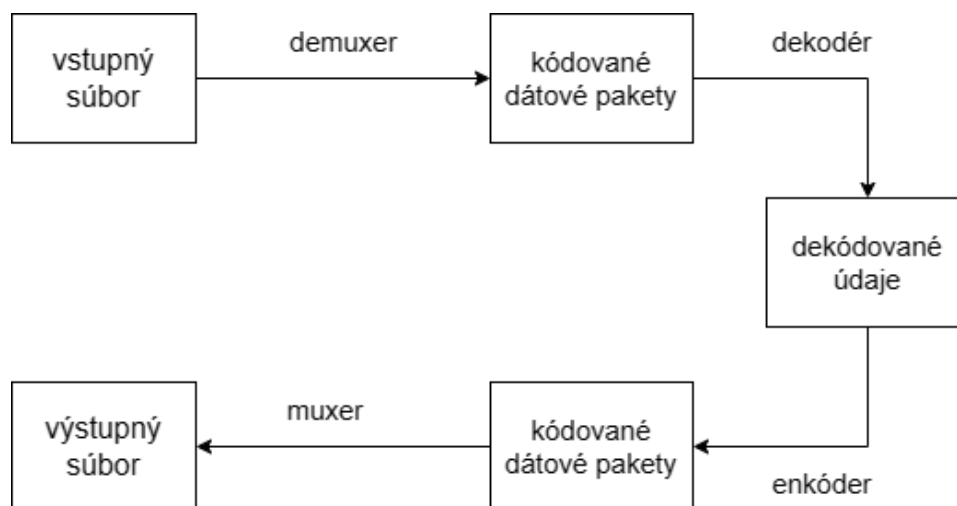
Pri nahrávaní skladby v telefóne sa zvukové vlny vášho hlasu pomocou Analog-to-Digital Converter (ADC) konvertujú na digitálny zvukový signál. Digitálny zvukový signál sa potom uloží na pamäťovú kartu telefónu. Pri prehrávaní skladby

¹Prevzaté z [2] (preložené z angličtiny)

sa digitálny zvukový signál prevedie späť na elektrický signál pomocou Digital-to-Analog Converter (DAC). Elektrický signál sa potom zosilní a prehráva cez reproduktory telefónu.

1.2 ffmpeg

ffmpeg[3] je multimediálny rámec, ktorý pracuje so zvukom a videom a má mnoho schopností na kódovanie, dekódovanie, streamovanie, prehrávanie obsahu a ďalšie. Je multiplatformový, takže náš kód, ktorý ho bude používať, bude fungovať rovnako ako v systémoch Windows, Linux, Mac OS X a ďalších. Obsahuje tiež mnoho knižníc, takže by mal podporovať čo najviac formátov.



Obr. 1.2: Proces transkódovania pomocou ffmpeg²

1.3 Programovací jazyk Python

Python je univerzálny programovací jazyk, ktorý sa hodí na rôzne úlohy, napríklad na analýzu zvuku. Tento jazyk má mnoho výhod: ľahko sa učí, je čitateľný, výkonný a má výkonné knižnice. Zároveň má aj nevýhody: je pomalý, pretože je to interpretovaný jazyk, spotrebuje viac pamäte, lebo na správu pamäte používa garbage collector.

²Prevzaté z [4] (preložené z angličtiny)

1.3.1 Knižnice pre vedecké výpočty a analýzu zvuku

- NumPy[5] a SciPy[6] sú základné knižnice jazyka Python na vedecké výpočty. Poskytujú vysoko výkonný objekt viacrozmerného poľa a súbor procedúr na prácu s takýmito poľami. Tieto objekty sú efektívne a ľahko použiteľné a možno ich použiť na reprezentáciu širokej škály údajov, ako sú obrázky, zvukové signály a vedecké údaje.
- librosa[7] je knižnica jazyka Python na analýzu hudby a zvuku. Poskytuje základné bloky potrebné na vytvorenie systémov na vyhľadávanie hudobných informácií a komplexnú sadu funkcií na analýzu a manipuláciu so zvukovými signálmi. Librosa je postavená nad NumPy a SciPy a poskytuje vysokoúrovňové rozhranie na vykonávanie úloh analýzy zvuku.

1.3.2 Knižnice pre strojové učenie

- TensorFlow[8] je softvérová knižnica s otvoreným zdrojovým kódom pre strojové učenie a umelú inteligenciu. Poskytuje komplexný ekosystém nástrojov, knižníc a komunitných zdrojov, ktoré umožňujú výskumníkom a vývojárom jednoducho vytvárať a nasadzovať aplikácie využívajúce strojové učenie.
- PyTorch[9] je populárna knižnica strojového učenia, ktorá poskytuje dve funkcie na vysokej úrovni: tenzorové výpočty so silnou akceleráciou na GPU a hlboké neurónové siete. PyTorch sa široko používa na aplikácie, ako je počítačové videnie a spracovanie prirodzeného jazyka.

1.3.3 Iné knižnice

- SpeechRecognition³ je knižnica na rozpoznávanie reči, ktorá dokáže rozpoznať vstup z mikrofónu, konvertovať zvuk na text a ďalšie, podporuje populárne enginy ako Google, Microsoft, Whisper a podobné.
- shazamio⁴ je asynchrónna knižnica, ktorá bola vytvorená reverzným inžinierstvom aplikácie Shazam na rozpoznanie skladby, poskytnutie informácií o interpretovi, top skladbách krajín a viac.

³https://github.com/Uberi/speech_recognition

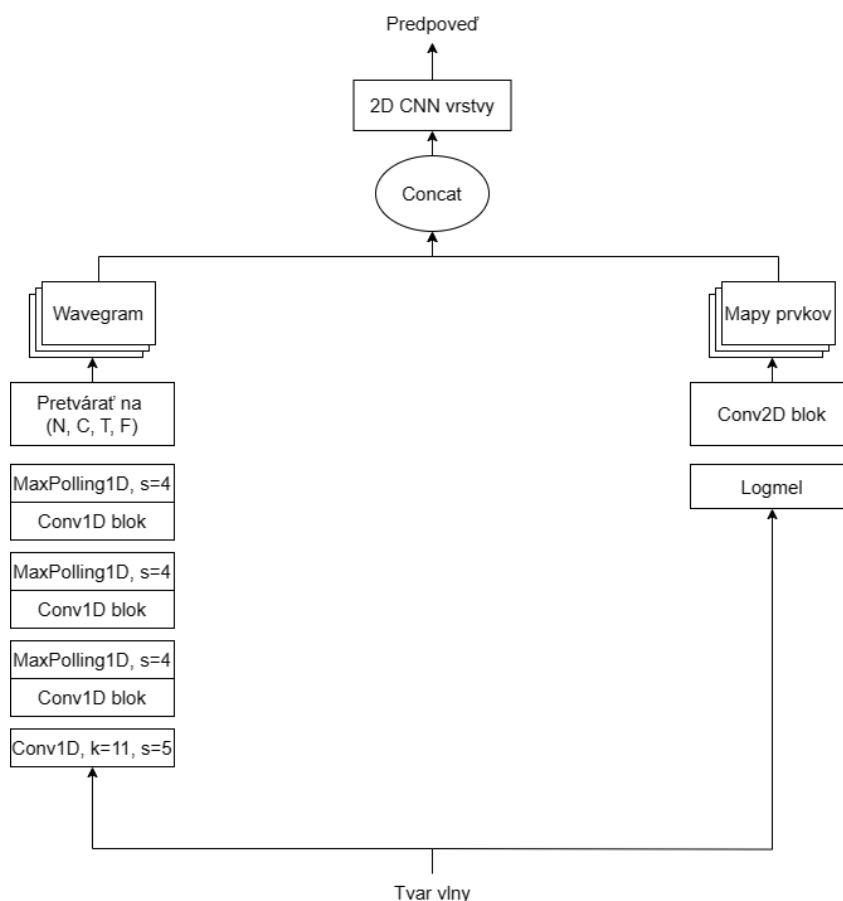
⁴<https://github.com/shazamio/ShazamIO>

- Flask⁵ je rámec webových aplikácií “Web Server Gateway Interface” v jazyku Python, ktorý sa ľahko používa a na jeho používanie nie sú potrebné externé nástroje alebo knižnice.

1.4 PANNs

Pretrained Audio Neural Networks (PANNs)[10] je architektúra neurónovej siete, ktorá je vopred natrénovaná na rozsiahlom súbore údajov AudioSet[11], čo je súbor zvukových udalostí označených človekom. Architektúra PANNs je navrhnutá na vykonávanie úloh rozpoznávania zvukových vzorov, ako je označovanie zvuku, klasifikácia hudby a detekcia zvukových udalostí. Architektúra PANNs je modelovaná rôznymi Convolutional Neural Networks (CNN) a ako vstupné prvky používa log-mel spektrogram aj tvar vlny

PANNs používa Wavegram-Logmel-CNN na klasifikáciu zvuku.



Obr. 1.3: Algoritmus Wavegram-Logmel-CNN⁶

⁵<https://github.com/pallets/flask>

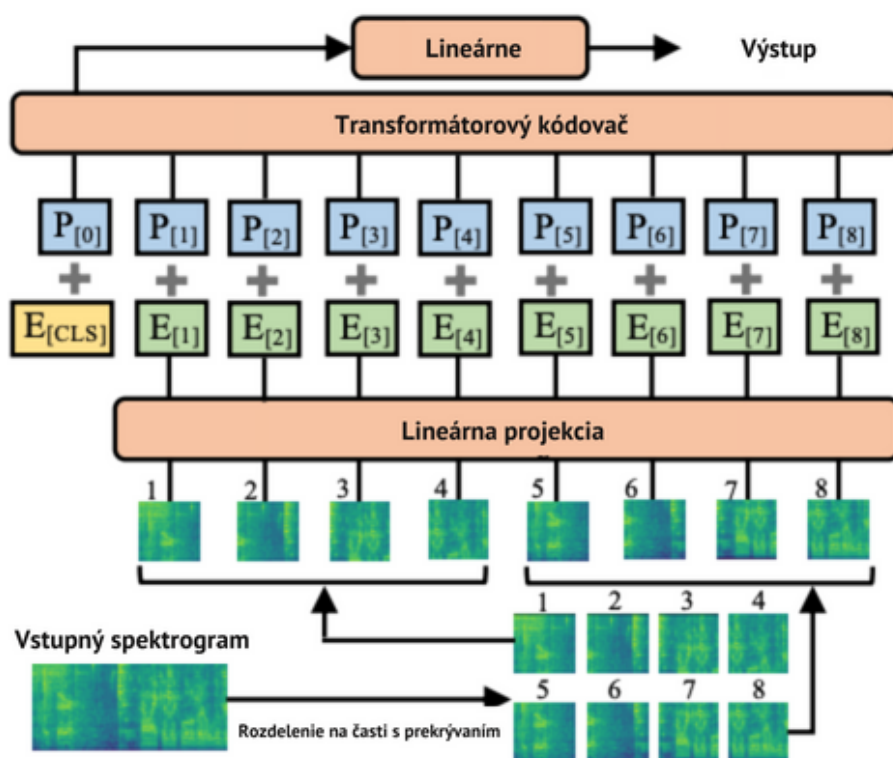
⁶Prevzaté z [10] (preložené z angličtiny)

Vlnková vetva siete získava vlastnosti zo surového tvaru vlny zvukového signálu. Robí sa to pomocou série konvolučných vrstiev. Log-melová vetva siete získava vlastnosti z log-melového spektrogramu zvukového signálu. Výstupy vetvy vlnového tvaru a log-melovej vetvy sa spoja dohromady. Konkaténovaný výstup sa privádza do série konvolučných vrstiev, po ktorých nasleduje plne prepojená vrstva, ktorá predpovedá výstupy. Konvolučné vrstvy sa učia extrahovať čoraz zložitejšie funkcie z konkaténovaného výstupu, zatiaľ čo plne prepojená vrstva vyprodukuje jednu predikciu pre zvukový signál.

1.5 AST: Audio Spectrogram Transformer

Audio Spectrogram Transformer (AST)[12] je model pre spracovania signálu, ktorá prevádza zvukový signál na vizuálnu reprezentáciu jeho frekvenčného obsahu v čase. Táto reprezentácia sa nazýva spektrogram a je užitočná na rôzne úlohy ako analýza zvuku, prepis hudby, rozpoznávanie reči.

Jednou z kľúčových vlastností AST je, že na klasifikáciu zvuku nepoužíva CNN, ale vlastný model.



Obr. 1.4: Algoritmus AST⁷

⁷Prevzaté z [12] (preložené z angličtiny)

Model funguje tak, že najprv konvertuje vstupný zvukový signál na spektrogram. Spektrogram sa potom rozdelí na postupnosť prekrývajúcich sa políčok, ktoré sa potom privedú do kódovača transformátora. Kódovač Transformer je typ neurónovej siete, ktorá sa dokáže naučiť závislosti s dlhým dosahom v sekvenciách údajov. To umožňuje transformátorovému kodéru naučiť sa globálne vzťahy medzi rôznymi časťami spektrogramu. Výstup kódovača Transformer sa potom privádza do lineárnej projekčnej vrstvy, ktorá vytvára konečný výstup modelu. Výstup modelu AST možno použiť na klasifikáciu zvukového signálu do rôznych kategórií.

1.6 musicnn

musicnn[13][14] je hudobne motivovaná CNN vycvičená na označovanie hudobných zvukov. Označovanie hudobného zvuku je úloha priradenia značiek k hudobnému zvukovému klipu, ako je žáner, nálada, nástroj, tempo atď. Skladá sa z front-endu CNN, ktorý je hudobne motivovaný, z husto prepojeného mid-endu a na výstupných vrstvách sa používa temporálne združovanie back-endu.

Prvým krokom je predspracovanie hudobných zvukových údajov. To zahŕňa normalizáciu zvukových úrovní a konverziu zvuku do formátu, ktorý môžu použiť CNN. Ďalším krokom je extrakcia vlastností zo zvukových údajov. To sa vykonáva pomocou rôznych techník, ako sú mel-frekvenčné cepstrálne koeficienty a spektrálne príznaky. Tieto vlastnosti zachytávajú rôzne zvuky, ktoré tvoria hudobnú skladbu. Extrahované vlastnosti sa potom vložia do CNN. Tieto siete sa naučia identifikovať vzory v znakoch, ktoré sú spojené s rôznymi hudobnými značkami. Výstupom CNN je vektor pravdepodobností, kde každá pravdepodobnosť predstavuje pravdepodobnosť, že sa v hudbe nachádza konkrétna značka. Posledným krokom je klasifikácia hudby na základe výstupu CNN. To možno vykonať pomocou rôznych klasifikačných algoritmov, ako sú napríklad stroje podporných vektorov a k-najbližších susedov. Výstupom klasifikačného algoritmu je značka pre hudbu, ktorá predstavuje najpravdepodobnejšiu značku pre hudbu.



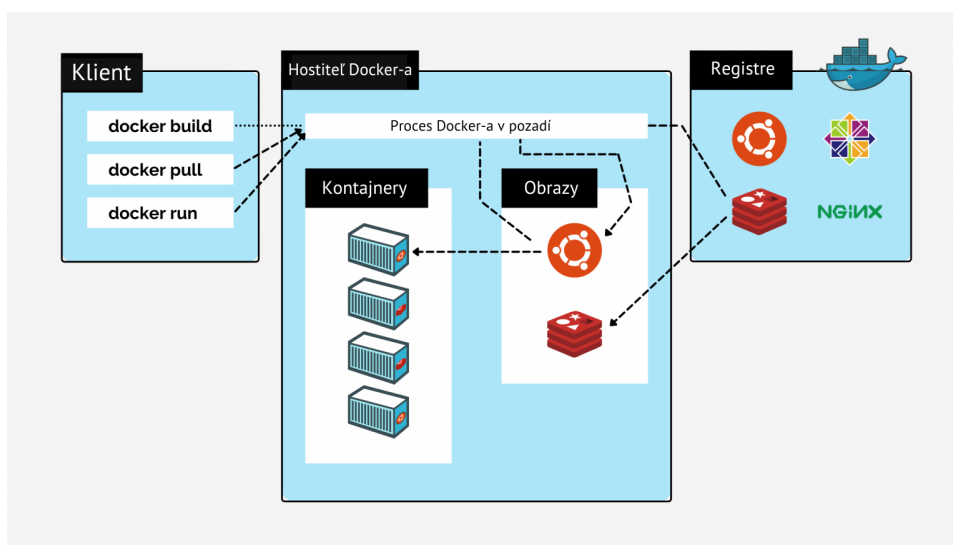
Obr. 1.5: Algoritmus musicnn⁸

⁸Prevzaté z [13] (preložené z angličtiny)

1.7 Docker

Docker[15] je platforma, ktorá nám umožňuje vytvárať kontajnerové aplikácie s naším kódom. Koncept platformy je podobný virtuálnemu počítaču, ale namiesto spustenia celého operačného systému používa jadro hostiteľského počítača a virtualizuje na softvérovej úrovni.

Jedným z dôvodov používania tejto technológie je vytváranie kontajnerov s rôznymi konfiguráciami, ktoré budú pracovať súčasne, nebudú si navzájom protirečiť a môžu byť spustené na ľubovoľných počítačoch.



Obr. 1.6: Architektúra Docker-a⁹

⁹Prevzaté z [16] (preložené z angličtiny)

2 Syntetická časť

Táto kapitola sa bude zaoberať implementáciou servera určeného na streamovanie zvuku z mikrofónov a klienta, ktorý bude tento stream prijímať, analyzovať a výsledky posielať brokerovi MQTT, a pokúsime sa náš systém kontanerizovať.

2.1 Implementácia servera

Momentálne nemáme prístup k výstupu z mikrofónov, preto implementujeme server, ktorý nám bude poskytovať stream zvuku. Na tento účel použijeme nástroje ako Flask[17], ffmpeg a pyaudio¹. Tento server môže byť užitočný nielen pre našu bakalársku prácu, ale aj pre ďalšie budúce projekty pre OpenLab.

2.1.1 Inicializácia servera

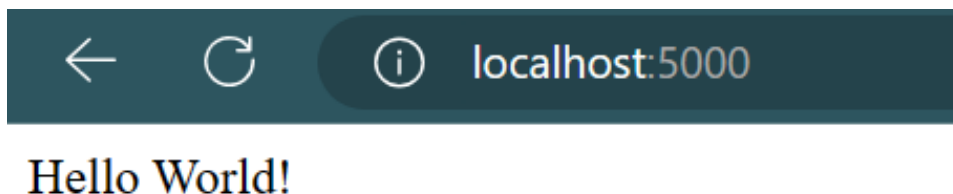
Pomocou Flasku inicializujeme úvodnú stránku, ktorá ukáže, že server funguje a slúži na ďalšie testovanie.

Zdrojový kód 2.1: Inicializácia skúšobnej stránky

```
1  from flask import Flask
2
3  @app.route('/')
4  def index():
5      return "Hello World!"
6
7  if __name__ == "__main__":
8      app.run(host='0.0.0.0')
```

Po spustení tohto kódu bude Flask používať dostupné adresy pre server a port 5000 ako predvolený. Môžeme sa zabezpečiť, že máme možnosť otvoriť testovaciu stránku tak, že prejdeme na odkaz localhost:5000, ak je všetko v poriadku, zobrazí sa stránka s textom “Hello World!”.

¹<https://people.csail.mit.edu/hubert/pyaudio/>



Obr. 2.1: Testovacia stránka

```
(ast_mus_str) PS D:\test> python .\server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://147.232.176.86:5000
Press CTRL+C to quit
127.0.0.1 - - [05/Apr/2024 19:30:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Apr/2024 19:30:10] "GET /favicon.ico HTTP/1.1" 404 -
```

Obr. 2.2: Flask po otvorení stránky

Na obrázku 2.2 je znázornené úspešné načítanie testovacej stránky zo servera, čo dokazuje výstup terminálu. To potvrdzuje dostupnosť servera a umožňuje používateľom zobraziť stránku v ich webovom prehliadači.

2.1.2 Parametre audia

Pred implementáciou dôležitých funkcií definujeme globálne premenné, ktoré budú popisovať naše parametre zvuku, ktorý sa bude streamovať:

- **FORMAT** (= `paFloat32`²) popisuje, aký dátový typ zvuku budeme používať a v našom prípade je to 32-bitový typ `floati`³,
- **CHANNELS** (= 1) je celočíselná premenná, ktorá opisuje počet zvukových kanálov, v našom prípade to znamená, že náš zvuk je monofónny,
- **CHUNK_SIZE** (= 1024) predstavuje počet vzoriek v bufferi. Uvádzame ho ako 1024, aby sme mali malé oneskorenie pri streamovaní zvuku,
- **SAMPLE_RATE** (= 16000) označuje počet vzoriek analógového zvukového signálu za sekundu, ktoré sa prekonvertujú do digitálneho formátu. Väčšina modelov, ktoré analyzujú zvuk, je trénovaná na zvuku so vzorkovacou frekvenciou 16000, preto používame toto číslo, aby sme mali presnejšie výsledky.

²Podporované typy nájdete v <https://people.csail.mit.edu/hubert/pyaudio/docs/#overview> (časť Portaudio Sample Formats)

³<https://www.sounddevices.com/32-bit-float-files-explained/>

2.1.3 Konfigurácia audio vstupu

V OpenLab máme dva mikrofóny, takže musíme vytvoriť konfiguračný súbor, aby sme zabezpečili, že môžeme spustiť dva servery s rôznymi mikrofónmi.

Konfiguračný súbor má len jeden reťazec, ktorý charakterizuje index zariadenia s mikrofónom a nazýva sa `device_id`. Do tejto premennej vložíme index, ktoré použije `pyaudio` na určenie výstupného mikrofónu. Nižšie môžete vidieť kód na získanie indexov vstupných zariadení.

Zdrojový kód 2.2: Kód na vyhľadávanie vstupných audio zariadení

```

1  import pyaudio
2
3  p = pyaudio.PyAudio()
4  info = p.get_host_api_info_by_index(0)
5  numdevices = info.get('deviceCount')
6
7  for i in range(0, numdevices):
8      if (p.get_device_info_by_host_api_device_index(0, i).get('
          ↪ maxInputChannels')) > 0:
9          print("Input Device id ", i, " - ", p.
              ↪ get_device_info_by_host_api_device_index(0, i).get('name'
              ↪ ))

```

Po spustení tohto kódu dostaneme tento výstup.

```

Input Device id  0  -  Microsoft Sound Mapper - Input
Input Device id  1  -  Microphone Array (Realtek(R) Au

```

Obr. 2.3: Výsledok s dostupnými vstupnými zvukovými zariadeniami

V našom prípade naše zariadenie používa mikrofón s indexom 1, takže toto číslo vložíme do konfiguračného súboru.

Po vytvorení konfiguračného súboru vytvoríme funkciu, ktorá uloží náš index zariadenia.

Zdrojový kód 2.3: Funkcia `read_config`

```

1  def read_config():
2      with open('config/config.json') as f:
3          config = json.load(f)
4      return config

```

Python má knižnicu, ktorá dokáže pracovať so súbormi JSON a my použijeme funkciu `load()`⁴, ktorá uloží náš súbor ako zoznam a môžeme ho použiť na spustenie servera s rôznymi mikrofónmi.

2.1.4 Získanie zvuku z mikrofónu

V ďalšom kroku implementujeme funkcie určené na zachytávanie zvukových údajov priamo z mikrofónov. Tieto funkcie potom spracujú zachytený zvuk prostredníctvom procesu prekódovania a nakoniec ho pripravia na prenos na server. Zdrojový kód implementujúci tieto funkcie je uvedený nižšie.

Zdrojový kód 2.4: Funkcie `response` a `read_audio`

```

1  def read_audio(inp, audio):
2  while True:
3      inp.write(audio.read(num_frames=CHUNK_SIZE))
4
5  def response():
6      a = pyaudio.PyAudio().open(
7          format=FORMAT,
8          channels=CHANNELS,
9          rate=SAMPLE_RATE,
10         input=True,
11         input_device_index=config['device_id'],
12         frames_per_buffer=CHUNK_SIZE
13     )
14
15     c = f'ffmpeg -re -f f32le -acodec pcm_f32le -ar {SAMPLE_RATE} -ac
16         ↪ {CHANNELS} -i pipe: -f mp3 pipe:'
17     p = Popen(c.split(), stdin=PIPE, stdout=PIPE)
18     Thread(target=read_audio, args=(p.stdin, a), daemon=True).start()
19
20     while True:
21         yield p.stdout.readline()
22
23 if __name__ == "__main__":
24     config = read_config()
25     app.run(host='0.0.0.0')
```

⁴<https://docs.python.org/3/library/json.html>

Ako vidíme v tomto zdrojovom kóde 2.4, funkcia `response()` inicializuje zvukový objekt pomocou `pyaudio` a spustí podproces `ffmpeg` na prekódovanie v reálnom čase. Ďalším krokom je vytvorenie vlákna, ktoré bude čítať zvuk z mikrofónu a pošle ho do podprocesu `ffmpeg` pomocou funkcie `read_audio`. Posledným krokom je vytvorenie nekonečnej slučky, ktorá poskytne zakódované zvukové údaje, aby bolo možné načítať stream klientom alebo inými aplikáciami.

2.1.5 HTTP Live Streaming

HTTP Live Streaming (HLS)[18] je protokol na poskytovanie zvuku a videa cez internet pomocou protokolu HTTP, ktorý používame na načítanie webových stránok. Tento protokol budeme používať na to, aby sme mohli streamovať zvuk a otvárať ho prostredníctvom prehliadača a/alebo prehrávačov, ktoré podporujú streamy HTTP.

Zdrojový kód 2.5: Funkcia `audio_feed`

```

1  @app.route('/audio_feed', methods=['GET'])
2  def audio_feed():
3      return Response(
4          response(),
5          headers={
6              'Cache-Control': 'no-cache, no-store, must-revalidate',
7              'Pragma': 'no-cache',
8              'Expires': '0',
9          },
10         mimetype='audio/mpeg')

```

```

[aist#0:0/pcm_f32le @ 00000291ffd4eb80] Gussed Channel Layout: mono
Input #0, f32le, from 'pipe:':
  Duration: N/A, bitrate: 512 kb/s
  Stream #0:0: Audio: pcm_f32le, 16000 Hz, 1 channels, flt, 512 kb/s
Stream mapping:
  Stream #0:0 -> #0:0 (pcm_f32le (native) -> mp3 (libmp3lame))
Output #0, mp3, to 'pipe:':
  Metadata:
    TSSE           : Lavf60.18.100
  Stream #0:0: Audio: mp3, 16000 Hz, mono, fltp
  Metadata:
    encoder       : Lavc60.35.100 libmp3lame
127.0.0.1 - - [05/Apr/2024 22:37:26] "GET /audio_feed HTTP/1.1" 200 -
|size=      34kB time=00:00:11.63 bitrate= 24.2kbits/s speed=1.02x

```

Obr. 2.4: Konverzia zvuku pre streamovanie pomocou `ffmpeg` v termináli

Zdrojový kód 2.5 definuje trasu URL `/audio_feed`, kde výstupom je kontinuálne konvertovaný zvuk, ktorý môžeme počúvať. V hlavičkách je tiež definované, že stream sa nebude ukladať do vyrovnávacej pamäte a typ média je `mpeg`[19], aby ho prehliadač mohol otvoriť ako audio. V súčasnosti takmer všetky webové prehliadače obsahujú prehrávače videa a zvuku, takže ak sa pokúsime vstúpiť na stránku, budeme môcť získať zvukový stream. Taktiež obrázok 2.4 nám zaručuje, že `ffmpeg` úspešne konvertuje zvuk a terminál zobrazuje proces prekódovania.

2.1.6 Vytvorenie kontajnera Docker so serverom

Nanešťastie sme nemohli vytvoriť kontajner Docker so serverom kvôli obmedzeniam aplikácie Docker. Jedným z problémov je prístup k mikrofónom z kontajnera, v súčasnosti môžeme vystaviť zariadenia len v prípade, že hostiteľský počítač a kontajner sú na Linuxe, to isté platí pre Windows, v inom prípade to nemôžeme urobiť, ak majú rôzne systémy⁵[20]. Druhým problémom bolo, že pri testovaní kontajnera v Ubuntu sme vystavili mikrofón kontajneru a spustili server, ale nemohli sme načítať zvukový stream a systém nevykazoval žiadne pokusy o prístup k mikrofónu.

2.2 Implementácia klienta

Hlavnou úlohou klienta bude prijímať zvuk z nami vytvoreného servera, klasifikovať zvukový vstup, ak ide o hlas, pokúsi sa ho previesť na text, ak ide o hudbu, pokúsi sa nájsť skladbu pomocou `shazamio`, analyzovať vlastnosti skladby a výsledky pošle do MQTT brokera `OpenLabu`.

2.2.1 Porovnanie AST a PANNs

Skôr ako začneme klient implementovať, musíme si vybrať, ktorý model bude fungovať rýchlejšie a/alebo presnejšie. Na túto úlohu sme použili zvukový záznam `papagája`, ktorý cvrliká a napodobňuje ľudskú reč. Každý model používa svoju najlepšiu metódu klasifikácie zvuku.⁶

⁵<https://docs.docker.com/reference/cli/docker/container/run/#device>

⁶Pre AST: mAP=0.4593, pre PANNs: mAP=0.438

Zdrojový kód 2.6: Výstup
AST

```

1 Domestic animals, pets: 0.591
2 Speech: 0.582
3 Bird: 0.373
4 Animal: 0.337
5 Dog: 0.251
6 Chirp, tweet: 0.093
7 Inside, small room: 0.079
8 Whistle: 0.075
9 Bird vocalization, bird call,
   ↪ bird song: 0.068
10 Squeak: 0.048
11 2.91s spent

```

Zdrojový kód 2.7: Výstup
PANNs

```

1 Domestic animals, pets: 0.621
2 Animal: 0.581
3 Speech: 0.526
4 Dog: 0.482
5 Chirp, tweet: 0.418
6 Bird: 0.392
7 Bark: 0.347
8 Bird vocalization, bird call,
   ↪ bird song: 0.255
9 Inside, small room: 0.072
10 Sneeze: 0.070
11 4.42s spent

```

Na základe týchto výsledkov vidíme, že AST klasifikoval lepšie ako PANNs, pretože pracuje rýchlejšie a presnejšie, zatiaľ čo druhý model je pomalší a má väčšiu šancu klasifikovať nesprávne. Preto sme si ako hlavnú klasifikáciu vybrali AST.

2.2.2 Konfiguračný súbor

Vytvoríme konfiguračný súbor aj pre klienta, aby sme mohli otvárať rôzne zvukové streamy, meniť jazyky na konverziu reči na text a jednoducho meniť brokery a témy MQTT. Kód na čítanie konfiguračného súboru je rovnaký ako pre server (zdrojový kód 2.3).

Zdrojový kód 2.8: Konfiguračný súbor pre klienta

```

1 {
2     "stream_url": "http://147.232.176.86:5000/audio_feed",
3     "duration": 30,
4     "lang_locale": "en-GB",
5     "mqtt_server": "mqtt.eclipseprojects.io",
6     "mqtt_port": 1883,
7     "general_topic": "bcthesis/general",
8     "music_topic": "bcthesis/music",
9     "transcribe_topic": "bcthesis/text"
10 }

```

- `stream_url` je odkaz na náš zvukový stream HLS,
- `duration` opisuje, koľko sekúnd bude nahrávať zo zvukového streamu,
- `lang_locale` určuje, do akého jazyka budeme konvertovať akúkoľvek reč. Jazyk by mal byť uvedený vo formáte "XX-YY", kde XX je dvojpísmenový kód z ISO 639⁷ a YY je z ISO 3166⁸,
- `mqtt_server` je odkaz na náš MQTT broker, tu používame verejný broker z Eclipse Foundation⁹ ako testovacie prostredie,
- `mqtt_port` je port na pripojenie k MQTT brokeru,
- `general_topic`, `music_topic` a `transcribe_topic` sú témy pre server MQTT. Prvá je určená na všeobecnú klasifikáciu zvuku, druhá na hudbu a posledná na konverziu reči na text.

2.2.3 Získanie zvukového streamu

Pred analýzou zvuku musíme zaznamenať stream zo servera a uložiť ho ako zvukový súbor. Na tento účel použijeme knižnicu `request`¹⁰ a `ffmpeg`.

Zdrojový kód 2.9: Funkcia main

```

1  if __name__ == "__main__":
2      config = read_config()
3      status_error = 0
4      hls_stream_url = config['stream_url']
5
6      while(True):
7          now = datetime.now()
8
9          output_file = now.strftime("audios/%Y%m%d_%H%M%S.mp3")
10
11         print(output_file)
12         duration_sec = config['duration']
13         if duration_sec < 10:
14             print("Duration can not be less than 10 seconds, please put
                ↪ the higher number in config.json")

```

⁷<https://www.iso.org/iso-639-language-code>

⁸<https://www.iso.org/iso-3166-country-codes.html>

⁹<https://iot.eclipse.org/projects/sandboxes/>

¹⁰<https://github.com/psf/requests>

```

15         exit(0)
16         download_hls_stream(hls_stream_url, output_file, duration_sec)

```

Pred spustením analýzy čítame konfiguračný súbor a získame aktuálny čas, ktorý použijeme ako názov súboru vo formáte RRRRMMDD_HH_MM_SS.mp3. Potom skontrolujeme, či je trvanie kratšie ako 10 sekúnd, ak je to pravda, zastavíme program a požiadame o zadanie vyššieho trvania, aby sme mali lepšie výsledky, a potom začneme nahrávať zvukový stream.

Zdrojový kód 2.10: Funkcia download_hls_stream

```

1  def download_hls_stream(url, output_file, duration_sec):
2      response = requests.get(url, stream=True)
3      if response.status_code != 200:
4          print(f"Failed to fetch the HLS stream. Status code: {response.
5              ↳ status_code}")
6          status_error += 1
7          if status_error == 3:
8              exit(0)
9          return
10     cmd = [
11         "ffmpeg", "-i", "-", "-vn", "-acodec", "copy", "-f", "mp3",
12         ↳ output_file
13     ]
14     process = Popen(cmd, stdin=PIPE)
15     start_time = time.time()
16     try:
17         for chunk in response.iter_content(chunk_size=1024):
18             if chunk:
19                 process.stdin.write(chunk)
20
21                 if time.time() - start_time > duration_sec:
22                     break
23     except KeyboardInterrupt:
24         exit(0)
25     finally:
26         process.stdin.close()
27         process.wait()
28     p = Process(target=analyze_audio, args=(output_file,))
29     p.start()

```

Zdrojový kód 2.10 sa pokúsi pripojiť k audio streamu, ak server nevráti kód 200, pokúsi sa pripojiť znova, ak sa to nepodarí ani na tretí pokus, program sa ukončí. Ďalším krokom bude spustenie podprocesu ffmpeg s parametrami, ktorý skopíruje kodek zo streamu bez videa, aby zabezpečil malú veľkosť súboru, a bude ho nahrávať, kým neprekročí čas trvania. Potom sa zaznamenaný súbor zanalyzuje na účely všeobecnej klasifikácie.

```
ffmpeg version 5.1.4-0+deb12u1 Copyright (c) 2000-2023 the FFmpeg developers
  built with gcc 12 (Debian 12.2.0-14)
  configuration: --prefix=/usr --extra-version=0+deb12u1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdi
r=/usr/include/x86_64-linux-gnu --arch=amd64 --enable-gpl --disable-stripping --enable-gnutls --enable-ladspa --enable-l
ibaom --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libcodecs2 --enable-
libdav1d --enable-libflite --enable-libfontconfig --enable-libfreetype --enable-libfribidi --enable-libglslang --enable-
libgme --enable-libgsm --enable-libjack --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt
--enable-libopus --enable-libpulse --enable-librabbitmq --enable-librist --enable-librubberband --enable-libshine --enab
le-lbsnappy --enable-libsoxr --enable-lspspeex --enable-lsrt --enable-libssh --enable-libsvtav1 --enable-libtheora --
enable-libtwolame --enable-libvidstab --enable-libvorbis --enable-libvpx --enable-libwebp --enable-libx265 --enable-libx
ml2 --enable-libxvid --enable-libzimg --enable-libzmq --enable-libzvt --enable-lv2 --enable-omx --enable-opengl --enabl
e-opencl --enable-opengl --enable-sdl2 --disable-sndio --enable-libjxl --enable-libpocketsphinx --enable-librsvg --enable-l
ibvpx --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-chromaprint --enable-frei0r --enable-libx264 --en
able-libplacebo --enable-librav1e --enable-shared
libavutil      57. 28.100 / 57. 28.100
libavcodec     59. 37.100 / 59. 37.100
libavformat    59. 27.100 / 59. 27.100
libavdevice    59.  7.100 / 59.  7.100
libavfilter    8. 44.100 /  8. 44.100
libswscale     6.  7.100 /  6.  7.100
libswresample  4.  7.100 /  4.  7.100
libpostproc   56.  6.100 / 56.  6.100
Input #0, mp3, from 'pipe:':
  Metadata:
    encoder      : Lavf60.18.100
  Duration: N/A, start: 0.000000, bitrate: 24 kb/s
  Stream #0:0: Audio: mp3, 16000 Hz, mono, fltp, 24 kb/s
Output #0, mp3, to 'audios/20240415_152202.mp3':
  Metadata:
    TSSE        : Lavf59.27.100
  Stream #0:0: Audio: mp3, 16000 Hz, mono, fltp, 24 kb/s
Stream mapping:
  Stream #0:0 -> #0:0 (copy)
size=      19kB time=00:00:06.55 bitrate= 24.3kbits/s speed=1.21x
```

Obr. 2.5: Nahrávanie streamu v klientovi pomocou ffmpeg

Obrázok 2.5 poskytuje potvrdenie úspešného spustenia procesu ffmpeg. Zachytený stream sa zaznamenáva bezchybne a bez akýchkoľvek komplikácií.

2.2.4 Odosielanie správ do MQTT

Predtým, ako implementujeme funkciu na analýzu zvuku, musíme vytvoriť funkciu, ktorá bude posilať naše správy na MQTT broker. Pre tento kód použijeme knižnicu paho-mqtt¹¹ od Eclipse Foundation.

Zdrojový kód 2.11: Funkcia send_to_mqtt

```
1 def send_to_mqtt(isAst, isMusic, res_list):
2     config = read_config()
3     json_data = json.dumps(res_list)
4     client = mqtt.Client()
5     client.connect(config['mqtt_server'], config['mqtt_port'])
6     if isAst:
7         client.publish(config['general_topic'], json_data)
8     elif isMusic:
```

¹¹<https://pypi.org/project/paho-mqtt/>

```

9         client.publish(config['music_topic'], json_data)
10    else:
11        client.publish(config['transcribe_topic'], json_data)
12    client.disconnect()

```

Kód 2.11 zobrazuje, že najprv prevedieme výsledky do formátu JSON, aby sme ich mohli odoslať na server. Potom sa pripojíme k MQTT brokeru, ktorý je uvedený v konfiguračnom súbore, a odošleme naše výsledky do jednej z tém, ktoré nám budú vyhovovať.

2.2.5 Analýza zvuku

Prvou analýzou bude všeobecná klasifikácia zvuku, ktorá nám pomôže dozvedieť sa o zvuku, a ak zistíme reč alebo hudbu, budeme ju analyzovať viac z druhej strany.

Zdrojový kód 2.12: Funkcia `analyze_audio`

```

1  def analyze_audio(output_file):
2      results = audio_inference.inference(output_file)
3      send_to_mqtt(True, False, results)
4      if 'Music' in results:
5          if results.get('Music') >= 0.5:
6              analyze_music(output_file)
7              return
8      if 'Speech' in results:
9          if results.get('Speech') >= 0.5:
10             trscrb_audio(output_file)
11             return

```

Autori AST poskytujú kód¹², ktorý môžeme použiť na analýzu zvuku na účely všeobecnej klasifikácie. Ako môžeme vychádzať z analytickej časti, kód prevedie zvuk na spektrogram a vloží ho do AST, potom model extrahuje zo spektrogramu vlastnosti, ktoré sú relevantné pre klasifikáciu, a potom predpovedá výsledky z natrénovaných údajov. Kód teda získa výsledky zo všeobecnej klasifikácie a ďalším krokom bude ich odoslanie na MQTT broker a kontrola, či máme predikciu hudby alebo reči väčšiu ako 0,5.

¹²<https://github.com/YuanGongND/ast/blob/master/egs/audioset/inference.py>

Zdrojový kód 2.13: Príklad správy zo všeobecnej klasifikácie

```

1  {
2      "file": "audios/20240414_235922.mp3",
3      "Finger snapping": 0.8133751749992371,
4      "Scissors": 0.06707838177680969,
5      "Chop": 0.05444669350981712,
6      "Inside, small room": 0.03961412236094475,
7      "Hands": 0.01584823988378048,
8      "Tick": 0.014201676473021507,
9      "Ping": 0.012325972318649292,
10     "Domestic animals, pets": 0.01103275828063488,
11     "Hiccup": 0.010426034219563007,
12     "Animal": 0.010108458809554577
13 }

```

Tu môžete vidieť príklad správy, ktorú dostaneme zo všeobecnej klasifikácie. Obsahom je názov súboru, ktorý bol analyzovaný, a desať najlepších predpovedí, ktoré budú vyhovovať tomuto zvuku.

Konverzia hlasu na text

Ak zistíme, že hlas ma predikciu viac ako 0.5 od všeobecnej klasifikácie, prevedieme zvuk na text, ak je to možné.

Zdrojový kód 2.14: Funkcia trscrb_audio

```

1  def trscrb_audio(output_file):
2      config = read_config()
3      temp_file = output_file[:-4] + '.wav'
4      cmd2 = [
5          "ffmpeg",
6          "-i", output_file,
7          temp_file
8      ]
9      process2 = Popen(cmd2, stdin=PIPE)
10     process2.wait()
11     text = speech_rec.transcribe(temp_file, config['lang_locale'])
12     send_to_mqtt(False, False, text)
13     os.remove(temp_file)

```


Zdrojový kód 2.14 otvorí podproces ffmpeg na prevod zvuku do formátu wav, pretože knižnica speechrecognition nepodporuje súbory mp3. Ďalším krokom je odoslanie zvuku do rozpoznávania reči Google s jazykom, ktorý je uvedený v konfigurácii. Po získaní výsledkov ich odošleme na MQTT broker a skonvertovaný súbor odstránime.

Príkladom správy bude názov zvukového súboru a text, ktorý bol prevedený z reči na text. V prípade, že sme nedostali transkribovaný text, správa bude obsahovať "can't transcribe or not detected" (nemožno transkribovať alebo nebolo zistené).

Vyhľadanie skladby a analýza jej vlastností

V prípade, že hudba bude mať predikciu vyššiu ako 0.5, ďalším krokom bude analýza vlastností skladby a pokus o vyhľadanie základných informácií prostredníctvom knižnice shazamio.

Zdrojový kód 2.15: Asynchrónna funkcia find_music

```

1  async def find_music(output_file):
2      shazam = Shazam()
3      out = await shazam.recognize_song(output_file)
4      return out

```

Táto funkcia odošle náš zvuk na servery Shazam a vráti odpoveď json, ktorá bude obsahovať výsledky nájdenej skladby. V tomto príklade 2.16 môžete vidieť príklad výsledku¹³, ktorý dostaneme.

Zdrojový kód 2.16: Príklad výsledku vyhľadania skladby

```

1  {
2      "matches": [
3          {
4              "id": "670634986",
5              "offset": 2.615604492,
6              "timeskew": 0.0033682585,
7              "frequencyskew": -0.00016111135
8          }
9      ],
10     "timestamp": 1710528022098,
11     "track": {
12         "type": "MUSIC",

```

¹³Väčšina reťazcov je odstránená z dôvodu zbytočných informácií

```

13     "title": "Sugarcoat (NATTY Solo)",
14     "subtitle": "KISS OF LIFE",
15     "genres": {
16         "primary": "K-Pop"
17     },
18 },
19 }

```

V prípade, že nájde skladbu, zobrazí nám interpreta, názov skladby a žáner, v inom prípade bude výsledok prázdny “matches”.

Zdrojový kód 2.17: Funkcia analyze_music

```

1  def analyze_music(output_file):
2      res = {'file': output_file}
3      info_track = asyncio.run(find_music(output_file))
4      if bool(info_track['matches']) == True:
5          res['artist'] = info_track['track']['subtitle']
6          res['title'] = info_track['track']['title']
7          res['genre'] = info_track['track']['genres']['primary']
8
9      res['tags'] = music_inference.inference(output_file)
10     send_to_mqtt(False, True, res)

```

Zdrojový kód 2.17 sa pokúsi nájsť skladbu z kódu 2.15, ak existuje zhoda, pridá do výsledkov interpreta, názov a žáner skladby. Ďalším krokom bude analýza vlastností piesne pomocou musicnn. Ono prevedie skladbu na číselnú reprezentáciu, ktorá je vhodná pre CNN, vloží ju do modelu, neurónová sieť extrahuje vlastnosti z údajov a výstupom budú vlastnosti ako hudobné nástroje, nálada alebo akékoľvek relevantné označenie. Posledným krokom bude odoslanie výsledkov do MQTT brokera.

Zdrojový kód 2.18: Príklad správy po analýze hudby

```

1  {
2      "file": "audios/20240416_231908.mp3",
3      "artist": "KISS OF LIFE",
4      "title": "Sugarcoat (NATTY Solo)",
5      "genre": "K-Pop",
6      "tags": ["electronic", "techno", "synth", "weird", "beat", "drums"
7              ↪ , "ambient", "fast", "no vocals", "guitar"]
8  }

```

Príklad 2.18 znázorňuje štruktúru správy v prípade úspešného vyhľadania skladby. V tomto scenári správa obsahuje podrobnosti, ako je názov súboru, meno interpreta, názov skladby, žáner a charakteristiky. Naopak, ak je vyhľadanie skladby neúspešné, správa bude obsahovať len názov súboru a príznaky.

2.2.6 Vytvorenie kontajnera Docker s klientom

Po implementácii servera môžeme vytvoriť kontajner Docker, ktorý bude fungovať bez ohľadu na to, kde bude nainštalovaný.

Zdrojový kód 2.19: Kód súboru Dockerfile

```
1 FROM python:3.7.17-slim
2 WORKDIR /app
3 COPY . /app
4 RUN apt-get update \
5     && apt-get install -y ffmpeg portaudio19-dev gcc libc-dev alsa-
6     ↪ utils libblas-dev liblapack-dev \
7     && rm -rf /var/lib/apt/lists/*
8 RUN pip install -r requirements.txt --no-deps --no-cache-dir
9 RUN pip install --no-deps --no-cache-dir torch==1.8.1+cpu torchvision
    ↪ ==0.9.1+cpu torchaudio==0.8.1 -f https://download.pytorch.org/
    ↪ whl/torch_stable.html
10 CMD ["python", "client.py"]
```

Vezmeme základ štíhleho obrazu Pythonu, aby sme zmenšili veľkosť kontajnera a skopírovali všetky súbory do pracovného adresára, potom aktualizujeme zoznamy balíkov a nainštalujeme potrebné balíky, ktoré bude Python a jeho knižnice používať, a vyčistíme zoznamy. Ďalším krokom bude inštalácia balíkov, ktoré sme vložili do requirements.txt, bez kontroly kompatibility závislostí a bez vyrovnávacej pamäte. Posledný reťazec v súbore znamená, že kontajner po spustení spustí nášho klienta.

3 Vyhodnotenie

Po implementácii servera na streamovanie zvuku a klienta na jeho analýzu môžeme konečne prejsť k vyhodnoteniu nášho systému. Nakoľko je relatívne komplikované získať prístup do prostredia OpenLab a nerušiť ostatných ľudí, testovanie systému sme vykonávali pomocou lokálneho počítača a mikrofónu.

3.1 Vyhodnotenie servera

Server úspešne streamuje zvuk, môžeme sa k streamu dostať cez prehliadač, prehrávač, ktorý podporuje HLS a nášho klienta, avšak boli zistené obmedzenia týkajúce sa nepretržitého streamovania a využitia kapacity počítača:

- súčasná implementácia iniciuje nový proces ffmpeg vždy, keď klient požiada o stream. Toto správanie sa odchyľuje od očakávania nepretržitého bežiacieho toku. Ideálna funkčnosť by zahŕňala jeden proces ffmpeg, ktorý by neustále zachytával a kodoval zvuk z mikrofónu, pripravený na doručenie každému klientovi, ktorý o to požiada. Z tohto problému vyplýva druhý problém,
- keď o stream požiadajú dvaja a viac klientov, server inicializuje nový ffmpeg pre každého z nich. Nakoľko môže dôjsť k zvýšenému využitiu procesora a pamäte, najmä pri spracovaní súbežných spojení, môže to byť neefektívne, potenciálne viesť k zníženiu výkonu a vytváraniu rôznych výstupov pre každého klienta.

Musíme spomenúť aj problém s vytvorením kontajnera servera, ktorý sme opísali v syntetickej časti. Zatiaľ čo Docker má obmedzenia na vystavenie zariadení pre kontajner, nemá zmysel to robiť, pretože to nebude fungovať v každom počítači a technológia stratí svoj pôvodný zmysel. Možno v budúcnosti, keď vývojári tento problém vyriešia, budeme môcť vytvoriť kontajner pre server.

3.2 Vyhodnotenie klienta

Klient predvádza dobrú prácu pri analýze zvuku z mikrofónu, úspešne využíva Audio Spectrogram Transformer na kategorizáciu zvuku na hudbu alebo reč alebo iné zvuky. Na základe klasifikácie systém využíva musicnn na analýzu hudobných prvkov a integráciu knižnice shazamio na vyhľadávanie hudobných informácií. V prípade reči sa pokúša o prepis reči na text. Súčasná implementácia sa však spolieha na nahrávanie krátkeho zvukového klipu pred spracovaním. Tento prístup prináša oneskorenie a narúša klasifikáciu zvuku v reálnom čase:

- proces nahrávania spôsobuje oneskorenie medzi zachytením zvuku a získaním výsledkov klasifikácie. To môže byť nežiaduce v scenároch, v ktorých sa vyžaduje okamžitá akcia na základe zvukového obsahu, ako napríklad prevod reči na text,
- požiadavka nahráť zvukový súbor pred analýzou bráni spracovaniu v reálnom čase. V ideálnom prípade by mal systém klasifikovať a potenciálne konať na základe zvukového streamu v čase, keď je prijímaný z mikrofónu, a poskytovať MQTT brokeru rýchlejšie výsledky.

Na riešenie tohto problému musíme prehĺbiť naše chápanie spracovania zvuku, preskúmať viac modelov, ktoré sú schopné analyzovať zvuk v reálnom čase, efektívne ich využívať a zvýšiť celkový výkon analýzy. Vykonaním týchto krokov môže klient vyrásť na skvelý nástroj na analýzu zvuku v reálnom čase.

3.2.1 Vyhodnotenie presnosti výslednej analýzy

Keďže náš klient dokáže vykonať nielen všeobecnú klasifikáciu zvuku, ale aj analýzu hudby a prevod reči na text, vyhodnotíme tieto tri úlohy.

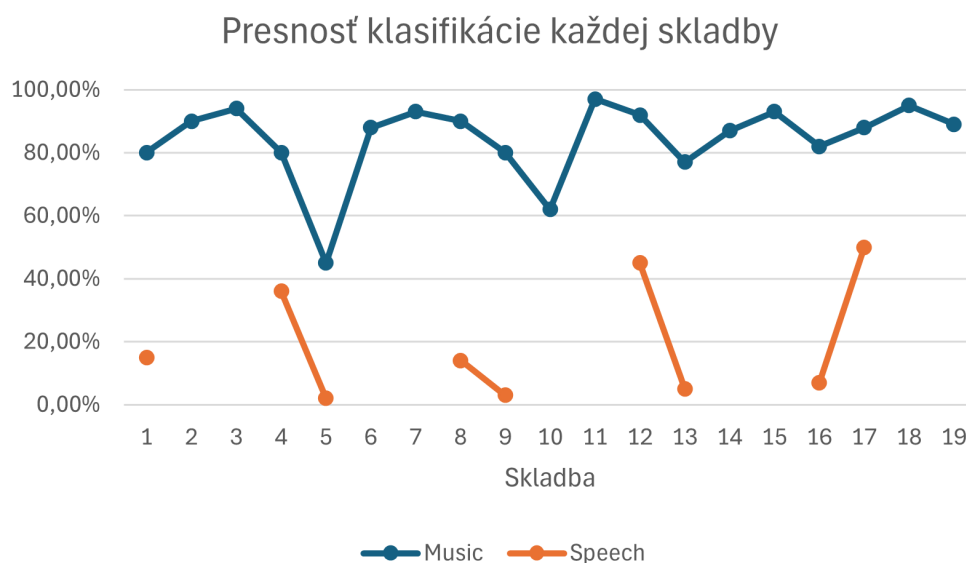
Presnosť všeobecnej klasifikácie

Pri tomto hodnotení sme imitovali zvuky a prehrávali ich, aby sme otestovali, ako klient vykoná všeobecnú klasifikáciu pomocou AST. Vo väčšine prípadov funguje dobre, najmä pri zvukoch ľudí a zvierat, niektorých hudobných nástrojoch a zvukoch, ktoré môžete počuť v domácnosti každý deň, priemerná presnosť bola medzi 85% a 95%. Niektoré zvuky ako tlieskanie boli mylne považované za krájanie potravín. Napriek tomu si myslíme, že klient robí všeobecnú klasifikáciu

úspešne, jedným z dôvodov je, že AST používa transformátory, ktoré sú vhodnejšie pre vstupy s premenlivou dĺžkou a zameriavajú sa na vzťahy medzi rôznymi časťami údajov, v tomto prípade je “údajom” spektrogram.

Presnosť analýzy hudby

Pri tomto hodnotení sme použili 19 rôznych skladieb, ktoré boli vydané po poslednej aktualizácii AudioSetu¹, aby sme sa zabezpečili, že model nebude trénovaný na týchto dátach.



Obr. 3.1: Graf s presnosťou všeobecnej klasifikácie skladieb

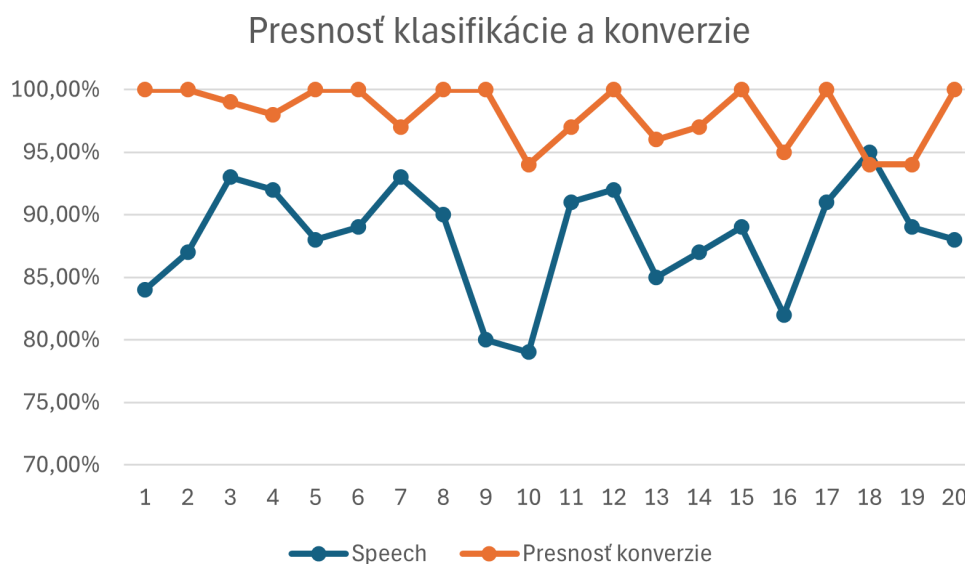
Ako vidíme na obr. 3.1, systém dosiahol vysokú presnosť pri klasifikácii testovaných audionahrávok ako hudby s priemerným percentom ~85%. Malá časť audionahrávok však bola nesprávne klasifikovaná ako reč alebo iné značky vrátane tých, ktoré opisovali hudobné nástroje. Vysoká presnosť pri rozpoznávaní hudby je celkovo sľubná. Skúmanie chýb nesprávnej klasifikácie však môže byť vodítkom pre ďalší vývoj s cieľom zlepšiť presnosť systému analýzy hudby.

Taktiež analýza vlastností hudby výborne predpovedala príznaky piesne, presné výsledky boli pri vokáloch a hudobných nástrojoch, ale nálada piesní nebola vždy správne zobrazovaná. Knižnica shazamio tiež funguje dobre a väčšina skladieb bola identifikovaná správne.

¹Posledná aktualizácia: 05.03.2017

Presnosť prevodu reči na text

Na vyhodnotenie presnosti prevodu reči na text sme zobrali 20 zvukových vzoriek, z ktorých prvých 10 je v angličtine a ďalších 10 v slovenčine.



Obr. 3.2: Graf s presnosťou klasifikácie reči a konverzie na text

Ako vidíme, klient úspešne detekuje reč a rozpoznávanie reči od spoločnosti Google dokáže konvertovať anglickú a slovenskú reč na text. Ako však vidíme, slovenské vzorky sú prevedené menej správne ako anglické. Aby sme zmerali presnosť prevodu, rozdelili sme počet správnych slov na počet všetkých. Priemerná presnosť prevodu anglickej reči je 98,80%, slovenskej 97,30%. Celková priemerná predikcia reči je 88,20%.

4 Záver

Naším hlavným cieľom v tejto bakalárskej práci bolo analyzovať, ako môžeme spracovať prehrávaný zvukový obsah z prostredia OpenLab a na základe toho navrhnúť kontajnerový systém, ktorý bude tieto dáta spracovávať. Zistili sme, ako sa zvuk spracováva do digitálnej podoby, že existujú modely ako AST a PANNs a porovnali sme ich, aby sme zistili, čo nám bude lepšie vyhovovať a dozvedeli sme sa o Dockeri, ktorý nám pomôže vytvoriť kontajnerový systém.

Náš systém sa skladá z dvoch častí: servera a klienta, ktoré sú obe napísané v jazyku Python. Hlavným cieľom klienta je nahrávať zvuk z mikrofónu a vytvárať z neho stream, zatiaľ čo klient tento stream prijíma a analyzuje. Na začiatku urobí všeobecnú klasifikáciu, ktorá nám povie, aké zvuky vychádzajú z mikrofónu. Ďalší krok bude pochádzať z predpovedí, ak má reč vysokú predikciu, systém ju pošle na rozpoznávanie reči od spoločnosti Google a my dostaneme transkribovaný text, v inom prípade, ak je predpovedaná hudba, bude analyzovať vlastnosti piesne ako nálada, vokály, hudobné nástroje a pokúsi sa ju vyhľadať v Shazame. Po tom všetkom systém odošle výsledky do MQTT brokera OpenLabu.

Po implementácii sme vyhodnotili, ako presný je náš systém a či plní svoju úlohu. Všeobecná klasifikácia je vykonaná dobre, avšak niektoré zvuky, ktoré si môžu byť veľmi podobné, nebudú dokonale klasifikované. Hudba bola tiež úspešne rozpoznaná, jej vlastnosti boli analyzované správne a takmer všetky skladby boli nájdené v Shazame, takisto dobre si počíнал pri predpovedaní reči a jej prevode na text, presnosť bola vysoká, ale občas sme si ešte mohli všimnúť niektoré slová, ktoré boli vynechané alebo neboli správne.

Bohužiaľ, nie všetko bolo implementované dokonale kvôli súčasným obmedzeniam a nedostatočným znalostiam v niektorých oblastiach. Nemohli sme vytvoriť kontajner pre server, pretože Docker podporuje vystavenie zariadení len vtedy, ak je operačný systém ekvivalentný s operačným systémom kontajnera a my sme sa zasekli na probléme s prístupom k mikrofónu vo vnútri kontajnera. Taktiež sme mohli vylepšiť streamovanie implementáciou ako celodenné, aby sme minuli menej počítačových zdrojov a mali malú latenciu. Klient by sa tiež mohol

zlepšiť implementáciou analýzy zvuku v reálnom čase, aby bolo možné získať rýchlejšie výsledky. Súčasná implementácia nahráva zvuk pred jeho analýzou a v niektorých prípadoch nie je efektívna, napríklad pri konverzii reči na text.

Celkovo systém plní svoje úlohy a môže byť pre študentov veľmi užitočným nástrojom. Môžeme tiež podotknúť, že táto implementácia sa dá využiť nielen v rámci OpenLabu, ale aj v niektorých študentských kluboch ako Rádio 9.

Literatúra

1. WATKINSON, J. *Art of Digital Audio*. Taylor & Francis, 2013. ISBN 9781136117091. Dostupné tiež z: https://books.google.sk/books?id=WI4gJGEb_i0C.
2. *Ada computer science* [online]. [cit. 2024-04-02]. Dostupné z : https://adacomputerscience.org/concepts/sound_sound.
3. TOMAR, Suramya. Converting video formats with FFmpeg. *Linux Journal*. 2006, roč. 2006, č. 146, s. 10.
4. *ffmpeg* [online]. [cit. 2024-05-07]. Dostupné z : <https://ffmpeg.org/ffmpeg.html#Detailed-description>.
5. HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der; GOMMERS, Ralf; VIRTANEN, Pauli; COURNAPEAU, David; WIESER, Eric; TAYLOR, Julian; BERG, Sebastian; SMITH, Nathaniel J.; KERN, Robert; PICUS, Matti; HOYER, Stephan; KERKWIJK, Marten H. van; BRETT, Matthew; HALDANE, Allan; RÍO, Jaime Fernández del; WIEBE, Mark; PETERSON, Pearu; GÉRARD-MARCHANT, Pierre; SHEPPARD, Kevin; REDDY, Tyler; WECKESSER, Warren; ABBASI, Hameer; GOHLKE, Christoph; OLIPHANT, Travis E. Array programming with NumPy. *Nature*. 2020, roč. 585, č. 7825, s. 357–362. Dostupné z DOI: 10.1038/s41586-020-2649-2.
6. VIRTANEN, Pauli; GOMMERS, Ralf; OLIPHANT, Travis E.; HABERLAND, Matt; REDDY, Tyler; COURNAPEAU, David; BUROVSKI, Evgeni; PETERSON, Pearu; WECKESSER, Warren; BRIGHT, Jonathan; VAN DER WALT, Stéfan J.; BRETT, Matthew; WILSON, Joshua; MILLMAN, K. Jarrod; MAYOROV, Nikolay; NELSON, Andrew R. J.; JONES, Eric; KERN, Robert; LARSON, Eric; CAREY, C J; POLAT, İlhan; FENG, Yu; MOORE, Eric W.; VANDERPLAS, Jake; LAXALDE, Denis; PERKTOLD, Josef; CIMRMAN, Robert; HENRIKSEN, Ian; QUINTERO, E. A.; HARRIS, Charles R.; ARCHIBALD, Anne M.; RIBEIRO, Antônio H.; PEDREGOSA, Fabian; VAN MULBREGT, Paul; SCIPY 1.0 CONTRIBUTORS. SciPy 1.0: Fundamental Algorithms for

- Scientific Computing in Python. *Nature Methods*. 2020, roč. 17, s. 261–272. Dostupné z [doi: 10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
7. MCFEE, Brian; MCVICAR, Matt; FARONBI, Daniel; ROMAN, Iran; GOVER, Matan; BALKE, Stefan; SEYFARTH, Scott; MALEK, Ayoub; RAFFEL, Colin; LOSTANLEN, Vincent; NIEKIRK, Benjamin van; LEE, Dana; CWITKOWITZ, Frank; ZALKOW, Frank; NIETO, Oriol; ELLIS, Dan; MASON, Jack; LEE, Kyungyun; STEERS, Bea; HALVACHS, Emily; THOMÉ, Carl; ROBERT-STÖTER, Fabian; BITTNER, Rachel; WEI, Ziyao; WEISS, Adam; BATTENBERG, Eric; CHOI, Keunwoo; YAMAMOTO, Ryuichi; CARR, CJ; METSAI, Alex; SULLIVAN, Stefan; FRIESCH, Pius; KRISHNAKUMAR, Asmitha; HIDAKA, Shunsuke; KOWALIK, Steve; KELLER, Fabian; MAZUR, Dan; CHABOT-LECLERC, Alexandre; HAWTHORNE, Curtis; RAMAPRASAD, Chandrasekhar; KEUM, Myungchul; GOMEZ, Juanita; MONROE, Will; MOROZOV, Viktor Andreevitch; ELIASI, Kian; NULLMIGHTYBOFO; BIBERSTEIN, Paul; SERGIN, N. Dorukhan; HENNEQUIN, Romain; NAKTINIS, Rimvydas; BEANTOWEL; KIM, Taewoon; ÅSEN, Jon Petter; LIM, Joon; MALINS, Alex; HEREÑÚ, Darío; STRUIJK, Stef van der; NICKEL, Lorenz; WU, Jackie; WANG, Zhen; GATES, Tim; VOLLRATH, Matt; SARROFF, Andy; XIAO-MING; PORTER, Alastair; KRANZLER, Seth; VOODOOHOP; GANGI, Mattia Di; JINOZ, Helmi; GUERRERO, Connor; MAZHAR, Abduttayyeb; TODDRME2178; BARATZ, Zvi; KOSTIN, Anton; ZHUANG, Xinlu; LO, Cash TingHin; CAMPR, Pavel; SEMENIUC, Eric; BISWAL, Monsij; MOURA, Shayenne; BROSSIER, Paul; LEE, Hojin; PIMENTA, Waldir. *librosa/librosa: 0.10.1*. Zenodo, 2023. Ver. 0.10.1. Dostupné z [doi: 10.5281/zenodo.8252662](https://doi.org/10.5281/zenodo.8252662).
 8. MARTÍN ABADI; ASHISH AGARWAL; PAUL BARHAM; EUGENE BREVDO; ZHIFENG CHEN; CRAIG CITRO; GREG S. CORRADO; ANDY DAVIS; JEFFREY DEAN; MATTHIEU DEVIN; SANJAY GHEMAWAT; IAN GOODFELLOW; ANDREW HARP; GEOFFREY IRVING; MICHAEL ISARD; JIA, Yangqing; RAFAL JOZEFOWICZ; LUKASZ KAISER; MANJUNATH KUDLUR; JOSH LEVENBERG; DANDELION MANÉ; RAJAT MONGA; SHERRY MORE; DEREK MURRAY; CHRIS OLAH; MIKE SCHUSTER; JONATHON SHLENS; BENOIT STEINER; ILYA SUTSKEVER; KUNAL TALWAR; PAUL TUCKER; VINCENT VANHOUCHE; VIJAY VASUDEVAN; FERNANDA VIEGAS; ORIOL VINYALS; PETE WARDEN; MARTIN WATTENBERG; MARTIN WICKE; YUAN YU; XIAOQIANG ZHENG. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. 2015. Dostupné tiež z: <https://www.tensorflow.org/>. Software available from [tensorflow.org](https://www.tensorflow.org/).

9. PASZKE, Adam; GROSS, Sam; MASSA, Francisco; LERER, Adam; BRADBURY, James; CHANAN, Gregory; KILLEEN, Trevor; LIN, Zeming; GIMELSHAIN, Natalia; ANTIGA, Luca; DESMAISON, Alban; KÖPF, Andreas; YANG, Edward; DEVITO, Zach; RAISON, Martin; TEJANI, Alykhan; CHILAMKURTHY, Sasank; STEINER, Benoit; FANG, Lu; BAI, Junjie; CHINTALA, Soumith. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Dostupné z arXiv: 1912.01703 [cs.LG].
10. KONG, Qiuqiang; CAO, Yin; IQBAL, Turab; WANG, Yuxuan; WANG, Wenwu; PLUMBLEY, Mark D. PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition. *CoRR*. 2019, roč. abs/1912.10211. Dostupné z arXiv: 1912.10211.
11. GEMMEKE, Jort F.; ELLIS, Daniel P. W.; FREEDMAN, Dylan; JANSEN, Aren; LAWRENCE, Wade; MOORE, R. Channing; PLAKAL, Manoj; RITTER, Marvin. Audio Set: An ontology and human-labeled dataset for audio events. In: *Proc. IEEE ICASSP 2017*. New Orleans, LA, 2017.
12. GONG, Yuan; CHUNG, Yu-An; GLASS, James. AST: Audio Spectrogram Transformer. In: *Proc. Interspeech 2021*. 2021, s. 571–575. Dostupné z DOI: 10.21437/Interspeech.2021-698.
13. PONS, Jordi; SERRA, Xavier. musicnn: pre-trained convolutional neural networks for music audio tagging. In: *Late-breaking/demo session in 20th International Society for Music Information Retrieval Conference (LBD-ISMIR2019)*. 2019.
14. PONS, Jordi; NIETO, Oriol; PROCKUP, Matthew; SCHMIDT, Erik M.; EH-MANN, Andreas F.; SERRA, Xavier. End-to-end learning for music audio tagging at scale. In: *19th International Society for Music Information Retrieval Conference (ISMIR2018)*. 2018.
15. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux journal*. 2014, roč. 2014, č. 239, s. 2.
16. SARAN, Girdharee. *What is Docker? - Whizlabs Blog* [online]. 2023-11-22. [cit. 2024-04-02]. Dostupné z : <https://www.whizlabs.com/blog/what-is-docker/>.
17. GRINBERG, Miguel. *Flask web development: developing web applications with python*. " O'Reilly Media, Inc.", 2018.

18. PANTOS, Roger; MAY, William. *HTTP Live Streaming* [RFC 8216]. RFC Editor, 2017. Request for Comments, č. 8216. Dostupné z DOI: 10.17487/RFC 8216.
19. *MIME types (IANA media types) - HTTP* | MDN. 2023-12-18. Dostupné tiež z: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types.
20. SETHMANHEIM. *Devices in containers on Windows*. 2023. Dostupné tiež z: <https://learn.microsoft.com/en-us/virtualization/windowscontainers/deploy-containers/hardware-devices-in-containers>.

Zoznam skratiek

ADC	Analog-to-Digital Converter.
AST	Audio Spectrogram Transformer.
CNN	Convolutional Neural Networks.
DAC	Digital-to-Analog Converter.
HLS	HTTP Live Streaming.
HTTP	Hypertext transfer protocol.
MQTT	Message Queuing Telemetry Transport.
PANNs	Pretrained Audio Neural Networks.

Zoznam príloh

Príloha A Systémová príručka

Príloha B CD médium – záverečná práca v elektronickej podobe

A Systémová príručka

Táto systémová príručka opisuje, ako nasadiť implementovaný systém pre rôzne prostredia. Taktiež popisuje, čo sme použili ako vývojové prostredie a jeho technické špecifikácie.

A.1 Vývojové prostredie

Systém pre Openlab bol vytvorený na notebooku Lenovo LOQ15IRH8. Nižšie si môžete pozrieť jeho technické špecifikácie:

- **Operačný systém:** Windows 11 Pro
- **RAM:** 16 Gb
- **Procesor:** Intel Core i7-13700H 2.40GHz x64
- **Grafická karta:** NVIDIA GeForce RTX 4060 Laptop GPU

A.2 Nasadenie systému

V tejto kapitole nájdete návod na nasadenie servera a klienta systému. Pred nasadením si nezapodnajte stiahnuť repozitár¹.

A.2.1 Nasadzovanie servera

Pred nastavením servera sa uistite, že máte nainštalované najnovšie verzie programov Python a ffmpeg. Nižšie sú uvedené pokyny na nasadenie servera:

1. Otvorte terminál v priečinku `/code/server` a vykonajte príkaz na inštaláciu potrebných knižníc: `pip install -r requirements.txt`²,

¹<https://git.kpi.fei.tuke.sk/kpi-zp/2024/bp.nazar.aleksanych/code>

²V prípade používania systémov macOS alebo GNU/Linux si prečítajte kapitolu Installation na tomto odkaze: <https://pypi.org/project/PyAudio/>

2. Spustíte kód 2.2 v Pythone, aby ste našli id mikrofónu a vložili ho do konfiguračného súboru, ktorý nájdete v priečinku `config`,
3. Spustíte server príkazom `python server.py` a skontrolujete prístup pomocou jedného z odkazov, ktoré poskytol Flask.

Skúste sa dostať na stránku `/audio_feed` s pomocou prehliadača, ak sa otvorí prehrávač a môžete počuť sami seba, server je nastavený úspešne.

A.2.2 Nasadzovanie klienta

Na nasadenie klienta je potrebné nainštalovať najnovšiu verziu nástroja Docker³ a vykonajte pokyny z `README.md`⁴. Nižšie sú uvedené pokyny pre proces nasadenia:

1. Spustíte Docker a otvorte terminál v priečinku `/code/analyzis`,
2. Vytvorte obraz spustením príkazu `docker build -t <meno obrazu> .`
3. Nájdite konfiguračný súbor v `/config` a zmeňte premenné (prečítajte si podkapitolu 2.2.2), ako to bude vyhovovať vám
4. Spustením tohto príkazu vytvoríte kontajner klienta s konfiguračným súborom a pripojeným priečinkom na uloženie zvuku:

```
1 docker run --publish 5000:5000 --volume ${PWD}/config/config.  
    ↪ json:/app/config/config.json --volume ${PWD}/audios:/  
    ↪ app/audios --name <meno kontajnera> <meno obrazu>
```

Ak sa klient pripojil k serveru, dokáže bez problémov nahrávať zvuk a odosiela výsledky do MQTT brokera, potom ste klienta úspešne nasadili.

A.3 Popis zdrojového kódu

A.3.1 Funkcie servera

`def response()`

Táto funkcia inicializuje proces `ffmpeg` na konverziu surového zvuku na podporovaný typ na streamovanie a poskytne ho.

³V prípade, že používate systém Windows, nainštalujte WSL, aby ste mohli správne pracovať s aplikáciou Docker.

⁴https://git.kpi.fei.tuke.sk/kpi-zp/2024/bp.nazar.aleksanych/code/-/blob/main/analyzis/pretrained_models/README.md

def read_audio(inp, audio)

Táto funkcia číta vstup z mikrofónu a odosiela ho do procesu ffmpeg na prekódovanie.

def audio_feed()

Táto funkcia poskytuje stránku \audio_feed a vracia prekódovaný zvuk z mikrofónu, ktorý môžeme počúvať cez prehliadač alebo prehrávač.

A.3.2 Funkcie klienta**def download_hls_stream(url, output_file, duration_sec)**

Táto funkcia zachytí zadané trvanie zvukového streamu zo špecifikovaného odkazu, uloží záznam a spustí jeho analýzu.

def send_to_mqtt(isAst, isMusic, res_list)

Táto funkcia sa pripojí k MQTT brokeru a odošle správu do jednej z tém v závislosti od kontextu.

def analyze_audio(output_file)

Táto funkcia vykoná všeobecnú klasifikáciu pomocou AST, odošle výsledky na MQTT broker a ak má reč viac ako 0,5 predikcie, požiada o konverziu reči na text, v prípade, že má hudba viac ako 0,5 predikcie, odošle zvuk na ďalšiu analýzu.

def trscrb_audio(output_file)

Táto funkcia odošle zvuk do rozpoznávania reči Google a vráti text konverzovaného textu alebo správu, ktorá opisuje, že nedokázala rozpoznať reč alebo nemohla vykonať prevod.

def analyze_music(output_file)

Táto funkcia analyzuje príznaky nahranej hudby a pokúsi sa ju nájsť v službe Shazam, v prípade, že ju nájde, vráti názov skladby, interpreta a žáner, v inom prípade vráti iba príznaky.

A.4 Obmedzenia

Súčasná implementácia servera nedokáže spracovať nekonečný stream a keď sa k jednému streamu pripájajú dvaja alebo viacerí klienti, dochádza k vytváraniu ďalších podprocesov ffmpeg, čo môže spôsobiť zníženie rýchlosti odosielania a celkového výkonu.